

EduBra: educador de braille

Rafael de A. Fernandes * Arthur Heberle ** Luiz Correia ***

* Faculdade de Engenharia De Computação, Universidade do Tecnológica Federal do Paraná, PR, (e-mail: rafael.2024@alunos.utfpr.edu.br)

** Faculdade de Engenharia De Computação, Universidade do Tecnológica Federal do Paraná, PR, (e-mail: aheberle@alunos.utfpr.edu.br)

*** Faculdade de Engenharia De Computação, Universidade do Tecnológica Federal do Paraná, PR, (e-mail: luizcorreia@alunos.utfpr.edu.br)

Abstract: This article presents a proposal for a low-cost assistive device intended to help blind and visually impaired people in learning braille. The instrument converts digital texts into tactile braille representations using plastic pegs associated with servo motors, along with audio feedback. Controlled by a Raspberry Pi in its core, EduBra enables letter-by-letter braille reading with additional resources: adjustable reading speed and the possibility of pausing and replaying words.

Resumo: Este artigo apresenta uma proposta de dispositivo assistivo de baixo custo para auxiliar pessoas cegas ou com deficiência visual no aprendizado de braille. O aparelho converte textos digitais em representações táteis de braille por meio de pinos de plástico associados a servomotores, com reforço em áudio. Controlado por uma Raspberry Pi em seu interior, o EduBra permite a leitura letra por letra em braille com recursos adicionais: velocidade de leitura ajustável e a possibilidade de pausar ou reprisar palavras.

Keywords: braille; accessibility; Raspberry Pi; assisted learning; inclusion.

Palavras-chaves: braille; acessibilidade; Raspberry Pi; aprendizado assistido; inclusão.

1. INTRODUÇÃO

O braille é um sistema de escrita e leitura tátil para as pessoas cegas. Nomeado em homenagem ao seu inventor, o francês Louis Braille, ele consta do arranjo de seis pontos em relevo, dispostos na vertical em duas colunas de três pontos cada, na chamada "cela braille". A diferente disposição desses seis pontos permite a formação de até 63 combinações ou símbolos para representar diferentes caracteres individuais, como os exibidos na Figura 1, ou abreviaturas.(1).

O braille por extenso, caractere por caractere, é denominado grau 1; já o grau 2 é a forma abreviada, empregada para representar as conjunções, preposições, pronomes, prefixos, sufixos, grupos de letras que são comumente encontradas nas palavras de uso corrente, com o objetivo de reduzir o volume dos livros impressos nesse sistema. Uma série de abreviaturas mais complexas forma o grau 3, que necessita de um conhecimento profundo da língua, uma boa memória e uma sensibilidade tátil muito desenvolvida por parte do leitor cego(1).

Apesar do avanço de tecnologias assistivas para pessoas cegas ou com baixa visão, a alfabetização em braille continua sendo um fator essencial para a sua autonomia (3). Este projeto propõe o desenvolvimento de um sistema assistivo chamado Edubra, cujo objetivo é proporcionar

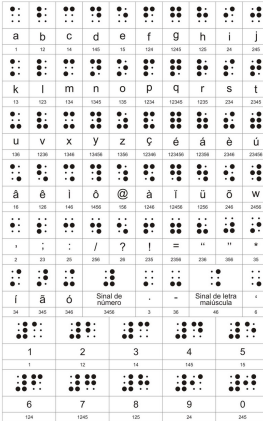
		Alfabeto Braille	
			
Cela Braille			
1	4		
2	5		
3	6		

Figura 1. Representação da cela braille com numeração de cada espaço, do alfabeto braille e de alguns caracteres comuns na língua portuguesa. De <https://movimentoescolainclusiva.blogspot.com>

uma ferramenta de aprendizado de braille de grau 1 de maneira interativa e acessível.

O site do Edubra transforma arquivos de texto digital em sinais táteis utilizando atuadores controlados por uma Raspberry Pi. Ao interpretar cada caractere, o sistema posiciona pinos em padrões braille correspondentes. Logo

antes disso, o som da letra é reproduzido via saída de áudio, facilitando a associação entre som e tato — estratégia eficaz para pessoas já alfabetizadas visualmente (5).

Além disso, o EduBra permite ao seu usuário interagir com o texto que está sendo reproduzido, de forma a melhorar sua experiência de aprendizagem. Ele pode ampliar ou abreviar o tempo de exposição dos caracteres e, consequentemente, a velocidade com que eles são trocados, de acordo com a sua proficiência na leitura, além de pausar esse processo ou voltar à palavra anterior quantas vezes desejar. Essa interação ocorre a partir de botões posicionados na tampa do gabinete com o objetivo de serem facilmente acionados pela mão do usuário que não estiver realizando a leitura.

A proposta visa não apenas facilitar o aprendizado do sistema braille, automatizando o apoio a quem está praticando a leitura, mas, também, torná-lo mais envolvente e adaptado à realidade individual do usuário, permitindo leitura de conteúdos personalizados como letras de músicas, artigos ou notícias (2).

2. TRABALHOS RELACIONADOS

Estudos como os de Alvaristo(2) e Batista(3) demonstram que o aprendizado do braille continua sendo indispensável para a independência de adultos com deficiência visual, apesar do surgimento de ferramentas digitais. A proposta do EduBra se alinha com essas diretrizes, mas explora a inovação por meio da integração entre tecnologia e práticas pedagógicas.

Além disso, a proposta se apoia em diretrizes técnicas estabelecidas pelo MEC(4), garantindo conformidade com normas de produção braille. A inspiração para a associação de múltiplos sentidos (audição e tato) no processo de aprendizado foi reforçada pelo trabalho de Melare(5), que analisa o papel das novas tecnologias no letramento de deficientes visuais.

3. METODOLOGIA

3.1 Componentes

- Raspberry Pi 4;
- 6 servomotores SG90;
- Pinos e célula braille impressas em PLA;
- Placa de solda PCB e solda;
- Potenciômetro Linear 10K;
- Módulo conversor analógico-digital ADS1115;
- 4 botões "push-button";
- Cabo Extensor P2;
- Fonte Chaveada 5V 10A 50W Bivolt;
- Fios, cabos e jumpers.

3.2 Raspberry

O EduBra demanda um sistema capaz de realizar simultaneamente o controle preciso de múltiplos servomotores, o processamento de sinais de entrada vindos de botões e potenciômetro, além de conectar-se à internet para o recebimento de arquivos e posterior conversão desses em comandos motores e saída de áudio. Por essas razões, o projeto exige um dispositivo mais robusto do que microcontroladores tradicionais, como Arduino ou ESP32, que,

embora adequados para tarefas simples, não possuem o poder de processamento ou a infraestrutura necessária para o volume de dados e multitarefa exigidos.

A escolha recaiu sobre a Raspberry Pi 4, mostrada na Figura 2, devido à sua maior capacidade computacional. Este modelo possui processador Quad-Core, memória RAM expansível e suporte nativo a redes Wi-Fi, características fundamentais para o funcionamento do EduBra. Outro diferencial é o fato da Raspberry Pi operar com um sistema operacional próprio, o que a torna capaz de executar códigos de maneira independente, necessitando de um computador somente para o envio do texto(6).

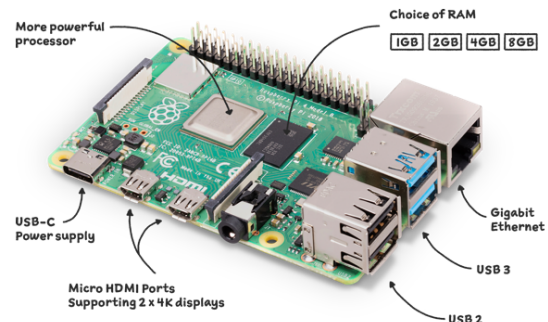


Figura 2. Visão do microprocessador, com elementos destacados. De <https://www.raspberrypi.com>

Além disso, o ecossistema da Raspberry Pi oferece suporte a diversas bibliotecas e frameworks, como GPIO (para controle de hardware) e ferramentas de comunicação em rede, essenciais para a arquitetura do EduBra. Isso possibilita integrar, em um único dispositivo, o recebimento de arquivos, a decodificação dos conteúdos em tempo real, a geração de sinais de controle para os motores, e a emissão de áudio com um só componente.

O fato de a Raspberry Pi ser um computador completo, e não apenas um microcontrolador dedicado, trouxe alguns desafios específicos durante a montagem e a configuração do projeto. Diferentemente de um microcontrolador convencional, como o Arduino, não é possível controlar a Raspberry Pi diretamente a partir de outro dispositivo sem realizar configurações prévias. Para evitar a necessidade de conectar um monitor, teclado e mouse diretamente à Raspberry Pi sempre que fosse necessário alterá-la ou configurá-la, foi preciso habilitar recursos de acesso remoto. Por meio da ativação dos protocolos SSH (Secure Shell) e VNC (Virtual Network Computing), tornou-se possível acessar e operar a Raspberry Pi integralmente a partir de outro computador, desde que ambos estejam conectados à mesma rede Wi-Fi(7).

Além disso, foi necessário adquirir um conversor de vídeo HDMI para VGA, mostrado na Figura 3, de modo a permitir a conexão da Raspberry Pi a monitores mais antigos que não possuem entrada HDMI, realidade comum nos ambientes de laboratório utilizados pela equipe. Essa etapa foi indispensável, uma vez que é necessário acessar a interface gráfica da Raspberry Pi para conectá-la a uma rede WI-FI pela primeira vez. Após essa configuração, ela passa a se conectar automaticamente à internet sempre que estiver dentro do alcance da rede previamente cadastrada,

permitindo seu controle remoto sem a necessidade de tela ou periféricos adicionais.



Figura 3. Módulo conversor entre dois padrões de sinal: VGA, analógico, e HDMI, digital. De <https://www.magazineluiza.com.br>

3.3 Pinos móveis

O EduBra reproduz, no dedo do usuário apoiado na cela braille, os sinais correspondentes a cada caractere do texto que está sendo soletrado. Para tal, utiliza 6 pinos móveis, cujas dimensões e formato da ponta buscam reproduzir com o máximo de fidelidade as normas técnicas do sistema aprovadas pelo MEC(4) para um melhor aprendizado. Cada um desses pinos está associado a um servomotor SG90 controlado pela RaspberryPi 4.

Tanto a cela como os pinos foram desenhados utilizando o programa SolidWorks, com licença cedida pela UTFPR e auxílio dos membros do seu projeto de extensão "O Projeto". Foram criados quatro diferentes modelos de pinos até alcançar a versão definitiva, exibida na Figura 4, usando diferentes formas de construção no SolidWorks, como cortes extrudados a partir de um paralelepípedo e a criação de sólidos de revolução interligados, na procura da peça que apresentasse melhor durabilidade mecânica, e melhor capacidade de integração aos servomotores, convertendo seu movimento rotacional em linear (vertical).

Já para a impressão em 3D dos pinos, foram usados, em diferentes tentativas, o espaço maker do "O Projeto", no Campus Centro, e o laboratório de prototipagem da UTFPR "PROTO", no Campus Ecoville, com seus computadores, programas fatiadores ("slicer") e impressoras. O filamento escolhido foi o polilático (PLA), também cedido pelo "PROTO". Devido às dimensões reduzidas da peça e seu formato arredondado, para o melhor resultado foi preciso configurar especialmente a impressora Ender 3MAX Neo do "PROTO"(9), a fim de maximizar a resolução da impressão.

Cada um dos pinos está preso a uma hélice de plástico, que por sua vez está ligada aos servomotores SG90 (8). Essas hélices, que foram compradas junto com os servomotores, tiveram seus furos alargados a fim de que a parte horizontal



Figura 4. Modelo final de pino. As junções em curvas suaves entre as partes da peça de diferentes dimensões diminuem as chances de quebra no que, sem elas, seriam pontos frágeis. Dos autores.

dos pinos pudesse passar com folga e ser presa do outro lado com cola quente num sistema semelhante a porcas e parafusos, isso é, permitindo que o pino rotacione independentemente da hélice e se mantenha sempre perpendicular à cela. Os servomotores estão fixados à parte interna da tampa do gabinete, circundando a cela de forma que seus respectivos pinos se encaixem nos espaços dessa, conforme a Figura 5.

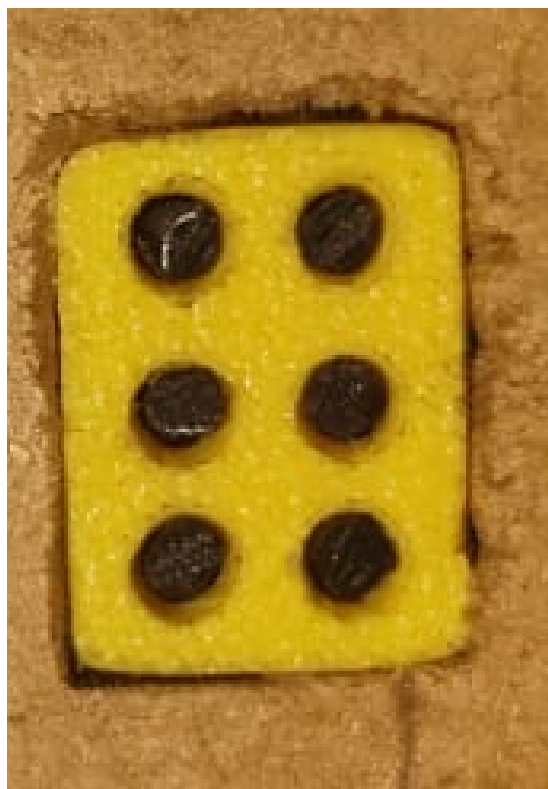


Figura 5. Vista superior da cela (em amarelo) com os pinos encaixados em seu espaço. Quando ativados, eles sobem, permitindo a leitura do caractere em braille.

Quando esses servomotores são acionados, as hélices giram e os pinos sobem, emergindo da cela de forma que o usuário possa senti-los e ler o caractere correspondente antes que esses sejam recolhidos para que um novo caractere

seja projetado. Uma trava física, feita de MDF, como o gabinete, foi posicionada abaixo dos pinos, garantindo que esses não venham a sair de suas posições por um impacto no gabinete ou um movimento impreciso dos servomotores no recolhimento. A Figura 6 mostra a montagem.



Figura 6. Visão do interior do gabinete, com os 6 servomotores posicionados de forma que os pinos (em preto) fiquem tão perpendiculares quanto possível e a trava de MDF. Dos autores

A equipe optou pelo servomotor sobre outros tipos de motor disponíveis pela sua capacidade de rotacionar diferentes amplitudes com boa repetibilidade e pela sua controlabilidade simples em microcontroladores. Dentre os servomotores, o modelo SG90 (8) foi escolhido pelo seu tamanho, peso e torque adequados, além de seus exemplares terem baixo custo e alta disponibilidade, o que permitiria substituí-los de forma simples. De fato, por erros de montagem e teste ao longo do projeto que levaram a curto-circuitos e ligações invertidas entre fonte e neutro, oito servomotores se avariaram e tiveram de ser descartados.

Uma das dificuldades iniciais do projeto foi a alimentação dos servomotores, pois, após pesquisar nos datasheets da Raspberry (6) dos servomotores e em fóruns, o grupo percebeu que não seria possível garantir que todos os servomotores recebessem da Raspberry a corrente elétrica para uma ativação quase simultânea, por limitações do microcontrolador e dos fios. Além disso, haveria riscos de sobrecarga na Raspberry, pois ela seria reponsável por fornecer energia e sinal de ativação para todos os servomotores, além de lidar com as demais funcionalidades do projeto.

A solução encontrada foi usar uma fonte chaveada para transformar a tensão da rede elétrica (seja ela 127V ou 220V) e fornecer, separadamente, a diferença de potencial para o funcionamento do microcontrolador e dos motores, 5,1V (6) (8). O modelo de fonte usado tem 7 chaves: um par positivo-negativo de alimentação, dois pares do mesmo tipo de saída e um neutro de alimentação (não utilizado). Esse componente fica dentro do gabinete, garantindo sua proteção e facilitando o transporte de todo o projeto, e apenas seu cabo de alimentação, do tipo rabicho, fica no exterior do gabinete.

Passando à conexão dos demais componentes com a fonte, mostrada na Figura 7, a Raspberry possui um par de chaves positivo e negativo próprio, acessadas pelos fios separados do seu cabo de alimentação, e das chaves do

outro par saem dois fios capazes de conduzir altas correntes, conectados, cada um, a um trilho de solda em uma placa PCB, que se tornam, um, positivo, e, outro, negativo. Com isso, os servomotores, que tiveram seus fios de alimentação positivos (vermelho) e negativos (marrom) ligados em paralelo aos respectivos trilhos passaram a estar constantemente energizados, apenas esperando os sinais da Raspberry para serem acionados e, posteriormente, voltarem ao estado inicial em cada caractere em que fossem necessários. O fio de sinal (amarelo) de cada servomotor foi ligado a um pino de GPIO, usado como saída, diferente da Raspberry. Com essa configuração, tornou-se possível eliminar potenciais problemas de excesso de corrente elétrica na fiação e garantir a alimentação correta da Raspberry (visto que seu próprio carregador lhe fornecia 5V, ligeiramente abaixo da tensão ideal).

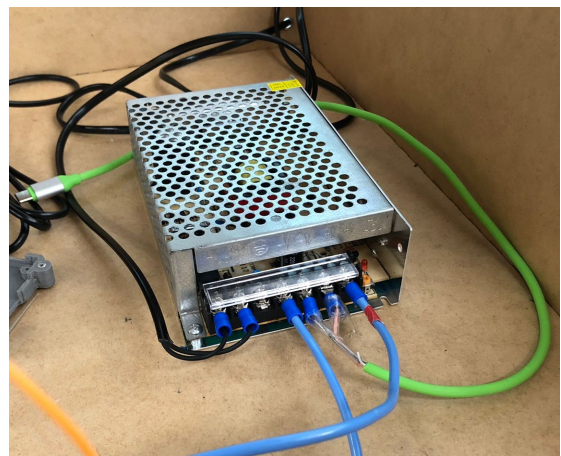


Figura 7. O cabo preto conecta os terminais de alimentação da fonte à rede externa; o cabo verde, bifurcado em uma chave positiva e uma negativa, fornece energia à Raspberry, enquanto os cabos azuis alimentam os motores. Dos autores.

Um problema enfrentado em uma fase posterior de testes foi o "tremor" de certos pinos, isso é, ocasionais e ligeiros movimentos de peças que não deveriam ser ativadas por problemas no envio de sinal por parte da Raspberry. Porém, esses tremores, que não puderam ser resolvidos, diferem significativamente dos reais movimentos dos pinos em termos de duração, pois são quase instantâneos, e de amplitude, que é muito baixa se comparada à apresentada na seção de resultados.

3.4 Botões, saída de áudio e regulador de volume

Cada um dos 4 botões reponsáveis por fazer o usuário controlar um aspecto da reprodução do texto recebeu um símbolo em alto relevo feito com cola quente, a fim de facilitar sua identificação por não videntes, como mostra a Figura 8. No interior do gabinete, cada botão foi soldado separadamente em um pedaço de placa PCB, de forma a, quando acionado, fechar nessa pequena placa um circuito entre o pino GPIO da Raspberry associado à sua função e um pino GND do microcontrolador, realizando no programa a ação desejada. Com o intuito de, nesse circuito, não usar jumpers, que ocasionalmente têm problemas de contato ou continuidade, ou soldar diretamente nos pinos de Raspberry, o que provavelmente os inutilizaria para usos

futuros, foi encaixada na coluna de pinos da Raspberry um extensor de empilhamento e, nos pinos desse extensor, foram soldados os fios que levam a cada pedaço de placa. A fim de que os botões sejam de fato comprimidos e, portanto, ativados (e não simplesmente movidos para baixo junto com gabinete, que naturalmente se deforma ligeiramente com a pressão do usuário), foram colocadas abaixo deles travas de MDF colado semelhantes às encontradas na região dos pinos móveis, de acordo com a Figura 9.



Figura 8. Botões na parte superior, com sinais relacionados à sua função: pause (cima), acelerar (direita), desacelerar (baixo) e voltar palavras (esquerda). Dos autores.



Figura 9. Os pedaços de PCB com os fios dos botões soldados, mantidos comprimidos pelas travas de MDF. Dos autores.

Os arquivos de áudio referentes às palavras e caracteres, respectivamente processados e armazenados pela Raspberry, são transformados em som pela mesma e emitidos pelo periférico ligado à sua saída P2 (6). Por razões de arquitetura do projeto no gabinete, pelo qual a Raspberry deveria estar em uma região central, em que pudesse atender com fios de menor comprimento tanto os pinos móveis quanto os botões, foi usado um cabo extensor P2 entre a saída do microcontrolador e a lateral do gabinete, onde o usuário pode conectar fones de ouvido, alto-falantes ou outro dispositivo de saída de som que desejar.

Enquanto os botões emitem inputs discretos à Raspberry, o regulador de volume deveria ser capaz de fornecer um sinal analógico, permitindo maior continuidade e, portanto,

mais opções de volume ao usuário. Para isso, foi escolhido um potenciômetro de ajuste giratório com capa plástica (knob), cuja interface simples, semelhante à de modelos de rádio e TV antigos, facilitaria a manipulação pelas pessoas a quem o projeto se destina.

Entretanto, a Raspberry não possui entradas analógicas, logo não foi possível conectar diretamente o potenciômetro a ela. A solução escolhida foi um conversor de sinais analógico-digital (ADC), do modelo ADS 1115, devido à disponibilidade. Para usá-lo, foi necessário entender conceitos do protocolo de comunicação I2C para realizar as conexões corretas entre o potenciômetro (emissor do sinal analógico), o conversor e o microcontrolador (receptor do sinal digital), conforme (11) e a Figura 10:

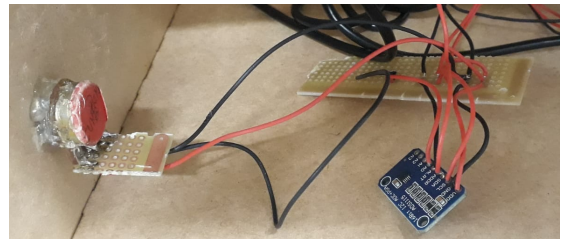


Figura 10. Imagem do interior do gabinete mostrando as conexões entre o potenciômetro e o conversor ADS1115. Fazer soldagens diretamente nesse componente sem fechar curtos revelou-se um desafio. Da placa PCB saem fios que se conectam com a Raspberry. Dos autores.

Além disso, foi preciso instalar, após muitas tentativas, as bibliotecas de software necessárias na Raspberry a fim de que ela reconhecesse o conversor e aceitasse seus sinais. Enfim, foi possível concluir a ligação entre todos os componentes de hardware, conforme a Figura 11.

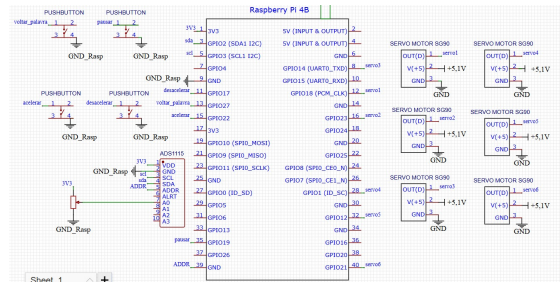


Figura 11. Esquemático elétrico do projeto, com as ligações entre o microcontrolador e os demais componentes. Dos autores

3.5 Gabinete

Por razões de leveza e custo, foram usadas para o gabinete chapas de compensado de madeira "medium density fiberboard" (MDF), de 3mm de espessura. Para alcançar as dimensões desejadas com precisão e fazer a montagem evitando o uso de ferramentas de marcenaria ou peças como cantoneiras e parafusos, que encareceriam o projeto e trariam dificuldades técnicas, o grupo optou por fazer as faces da caixa com cortes à laser e junções encaixáveis, conforme a Figura 12. Dessa forma, bastaria encaixar as faces adjacentes e aplicar cola para madeira a fim de montar a estrutura.

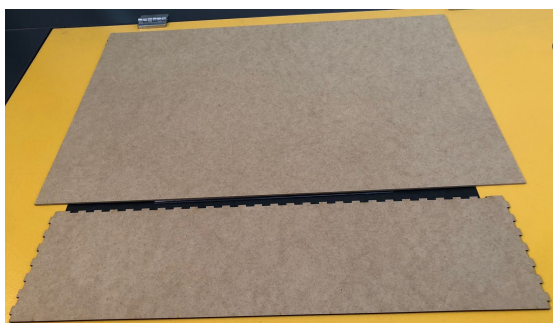


Figura 12. Chapa original de MDF e uma das faces com encaixes do gabinete produzida a partir dela. Dos autores.

O gabinete é formado por duas caixas abertas, isso é, sem fechamento superior: uma com maior profundidade que serve como fundo e outra, achatada e ligeiramente mais larga e comprida, que é a tampa quando virada ao contrário. O design inicial de ambas as partes foi feita no site MakerCase (10), que é capaz de gerar arquivos na extensão DXF do desenho de cada face a ser recortada, com dimensões, espessura do material e tamanho de junções personalizáveis. Esses arquivos podem ser posteriormente importados e editados em programas como AutoCad e SolidWorks. No caso do projeto, o último foi utilizado para desenhar na tampa do gabinete um furo retangular para a cela, conforme a Figura 13. É importante observar que outros furos poderiam ser feitos nos demais pontos do gabinete em que seria necessário uma ligação do exterior com os componentes internos, caso houvesse certeza prévia da posição deles. A Figura 14 mostra o design do fundo do gabinete.

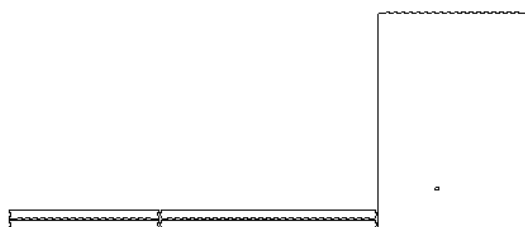


Figura 13. Visualização do arquivo em DXF da tampa do gabinete, com o furo para a cela. Dos autores.

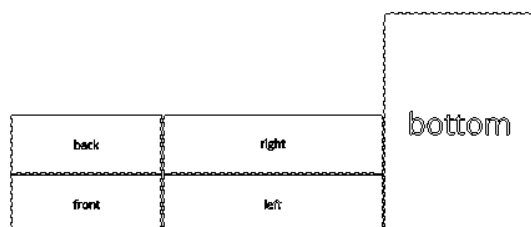


Figura 14. Visualização do arquivo em DXF do fundo do gabinete. Para melhor compreensão, foram deixadas as legendas de cada face, geradas automaticamente pelo MakerCase, que viriam a ser apagadas no software da máquina de corte a laser. Dos autores.

Levando as chapas de MDF ao laboratório "PROTO", foi possível, sob supervisão dos responsáveis, importar os arquivos contendo os desenhos das partes editadas do gabinete para o software específico da máquina CNC Router de Corte a Laser disponível. Nele, foi necessário selecionar quais das 10 faces (5 de cada parte) seriam cortadas em cada chapa de forma a aproveitar os recursos com o mínimo de desperdício e sem a necessidade de adquirir mais placas de MDF. Também foi preciso definir os limites ("frame") do corte para cada chapa, as posicionar de acordo na máquina e ajustar a intensidade do laser para a espessura do material. Além do corte, como a máquina possui um recurso adicional de gravação, foi possível estampar, nas faces correspondentes às laterais do gabinete, o nome do projeto (EduBra), os nomes abreviados dos integrantes da equipe e a instituição de ensino deles (UTFPR), como pode ser visto na Figura 15.

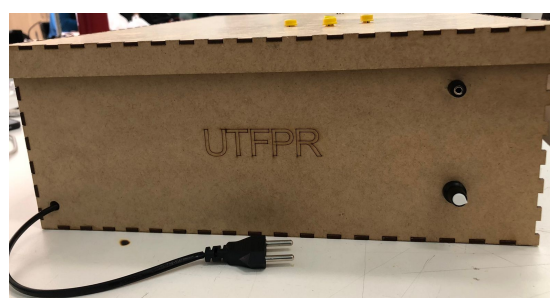


Figura 15. Lateral do gabinete onde foi gravada a sigla da instituição. Além dela, podem ser vistos o knob do regulador de volume, o cabo de alimentação do projeto, os botões que acionam as funções (em amarelo, no topo da imagem) e a entrada para fones de ouvido. Dos autores.

A montagem ocorreu de forma fácil, assim como fazer, com o auxílio de uma furadeira, os furos circulares para a ligação dos elementos ao exterior. Entretanto, por detalhes não levados em consideração, dois ajustes mais desafiadores tiveram de ser feitos. O primeiro, causado por um erro de medição anterior ao corte à laser, foi ampliar ligeiramente, nas duas dimensões, o espaço reservado à cela, usando uma serra tico-tico pertencente ao "O Projeto". O segundo, motivado por descon siderar a espessura do MDF, foi "afinar" a tampa do gabinete no local destinado aos botões, de forma que esses tivessem o espaço vazio entre sua base e seu topo necessário para suas molas se comprimirem e eles serem ativados. Para tal, foi necessário desgastar precisamente a tampa nessas regiões, com o uso de uma tupa laminadora, também de propriedade do "O Projeto".

3.6 Software

O código do projeto, produzido na linguagem de programação PYTHON em sua versão 3.12.10(12), consiste em dois arquivos: app.py e pi_reciever.py. O app.py atua no computador do usuário, criando nele um servidor, utilizando o framework Flask. Esse servidor web, utilizando um front-end feito em html(13), processa o upload de um arquivo (nos formatos .txt, .docx ou .pdf), extrai seu texto, insere as palavras deste texto em um array e, por fim, envia o texto e o array de palavras para a Raspberry Pi. No site gerado, cuja captura de tela é a Figura 16, há um campo

para inserir o arquivo com o texto e outro para enviá-lo para a Raspberry. Há ainda a opção de ajustar o intervalo entre cada letra exibida na cela do EduBra e, por fim, um botão para então iniciar a leitura na Raspberry.

Figura 16. Campos e botões do site gerado pelo Flask. Dos autores.

Já em `pi_receiver.py`, que é executado diretamente na Raspberry, o código consiste, majoritariamente, em duas principais funções ou rotas do próprio servidor (que também utiliza a biblioteca Flask). Ao início, são criadas as duas threads com o uso da biblioteca `threading`, uma para o site e outra para conferir o acionamento dos 4 botões no gabinete. A primeira thread recebe o texto e o array de palavras enviado pelo servidor Flask do `app.py`, salva-os na memória. Então, para fins de agilidade no momento de reproduzir as palavras, utilizando a biblioteca `gTTs`, da Google, cria arquivos de áudio referentes a cada palavra do array que tenha mais de um caractere. Isso porque, na memória da Raspberry, foi criada a pasta "letters", que contém arquivos de áudio no formato mp3 para cada letra, número ou caractere de uso corrente na língua portuguesa, de modo que não seja necessário criá-los a cada uso do EduBra (para fins de otimização).

A segunda rota, mais complexa, é inicializada após o usuário clicar no botão de início de leitura no site. Ela possui um loop, no qual percorre o array enviado pelo `app.py` e, a cada iteração, confere os botões para verificar se foram acionados e, portanto, se suas ações devem ser realizadas.

A biblioteca `pygame` é usada para reproduzir o arquivo mp3 e a biblioteca `smbus2` para receber o sinal digital do conversor analógico-digital, que se transforma no volume de saída do áudio. Assim, o arquivo de áudio de cada palavra é reproduzido no volume desejado e, após isso, cada um dos seus caracteres é percorrido, seu áudio é reproduzido e, por fim, os servomotores associados a eles são ativados.

Analisando os problemas enfrentados na confecção do código, o primeiro foi escolher a biblioteca para a saída de som. Inicialmente, foi testada a biblioteca `pyttxs3`, que era capaz de reproduzir palavras e letras de sem criar arquivos e sem conexão com a internet, mas o som não era claro o bastante para ser compreendido. Por isso, optou-se pela `gTTs`, que cria arquivos de áudio, reproduzidos posteriormente pelo `pygame`.

Além disso, houve complicações para processar a os sinais dos botões: devido ao tempo gasto para reproduzir os áudios das palavras e acionar os motores, o código não era percorrido com velocidade, de modo que, caso as instruções de leitura dos botões fossem inseridas simplesmente, essas não seriam alcançadas em tempo hábil. A conselho dos pro-

fessores da disciplina, foi usado o recurso de programação da interrupção para solucionar o problema, de forma que os botões eram lidos corretamente em qualquer mometo em que fossem ativados.

Apesar do sucesso inicial, essa solução deixou de ser viável depois que problemas de funcionamento da Raspberry obrigaram o grupo a reinstalar seu sistema operacional, o Raspberry Pi OS(6). Após isso, as interrupções deixaram de funcionar, pelo que foi feito um novo código usando duas "threads", conforme a Figura 17: uma delas fica responsável apenas pelas atividades do servidor e a outra monitora o acionamento dos botões. Essa executa um código principal que altera fatores na função que reproduz o texto, modificando, de acordo com os botões acionados, variáveis com o tempo entre as letras, a variável booleana que indica o pause, e o índice do array de palavras, usado para reprisar as palavras.

```
if __name__ == '__main__':
    try:
        start_pwm()
        print("[INFO] Starting flask server using thread")
        server_thread = threading.Thread(target=app.run, kwargs={"host": "0.0.0.0", "port": 5000})
        server_thread.daemon = True
        server_thread.start()

        button_thread = threading.Thread(target=monitor_buttons)
        button_thread.daemon = True
        button_thread.start()

        while True:
            time.sleep(1)
```

Figura 17. Função principal do código, que inicia as threads. Dos autores.

Por fim, bastou otimizar o código e aperfeiçoar empiricamente o funcionamento do EduBra mudando parâmetros definidos, como os ângulos correspondentes à rotação de cada servomotor, as velocidades máxima e mínima de reprodução, e o quanto essa muda a cada vez que os botões associados a ela são acionadas. Foi ainda alterada a correspondência entre os pinos GPIO da Raspberry e os acionadores e receptores, a fim de facilitar a integração final do projeto. O código completo se encontra em (14).

4. RESULTADOS

A avaliação correta dos caracteres na cela é subjetiva, dependendo muito da sensibilidade tátil do usuário e de sua experiência na leitura de braille. Portanto, a avaliação objetiva do desempenho do EduBra se baseou nos seguintes fatores:

- Precisão dos movimentos dos pinos móveis, medindo o quanto eles sobem em relação à cela quando ativados;
- Capacidade de adaptação à velocidade de leitura conforme o perfil do usuário, medindo o tempo mínimo entre a exibição de dois caracteres.

Para o primeiro item, cada pino, um vez levantado, foi medido com precisão de 0,1mm, usando a vareta de profundidade de um paquímetro. Para tal, foi usado o comando de pause no meio da reprodução de uma letra, de modo que o pino ficou travado em sua extensão máxima. Os resultados médios se encontram na Tabela 1:

Como observado a partir dos dados, os movimentos dos pinos alcançam uma altura que deve permitir a sua distinção, pelo usuário, dos tremores citados na seção 3.3. Além disso, uma vez que a diferença entre a altura dos pinos não passa de 0,3 mm, não deve haver impactos significativos na leitura, mas uma possível melhoria no

	Altura média (em mm)
Pino 1	2,7
Pino 2	2,4
Pino 3	2,6
Pino 4	2,4
Pino 5	2,4
Pino 6	2,6

Tabela 1. Altura média de cada pino. Eles foram numerados de acordo com a convenção dos espaços na cela braille, exposta na Figura 1. Dos autores.

projeto seria usar servomotores de outros modelos, ou um outro mecanismo de movimentação de pinos, que permita maior uniformidade.

Para a segunda avaliação, o botão de acelerar foi usado para que o EduBra operasse em sua velocidade máxima, ou seja, sem comandos de espera no execução do código. Nessas condições, foi medido o tempo entre o início da execução do áudio de uma letra e o áudio da seguinte, passando pelo acionamento e recolhimento dos pinos correspondentes. Como os pinos não são ativados simultaneamente, os tempos médios variaram de acordo com quantos pontos um dado caractere exige para ser representado. Para amostragem, foram selecionados caracteres com 1,2,3,4 ou 5 pontos associados e as medições foram realizadas com um cronômetro convencional, de celular, com precisão de 0,1 segundo. Seguem os resultados, na Tabela 2:

	Tempo médio (em segundos)
"a"(1 ponto)	2,6
"e"(2 pontos)	2,9
"l"(3 pontos)	3,1
"t"(4 pontos)	3,2
"y"(5 pontos)	3,5

Tabela 2. Relação entre caracteres com diferentes quantidades de pinos associados (ver Figura 1) e os respectivos tempos de exibição.

A partir da tabela, pode-se verificar, pela diferença entre os caracteres, que o tempo médio para a ativação e recolhimento de cada pino a mais é de 0,2 segundos, pelo que a maior parte do tempo gasto na exibição de um caractere se deve à biblioteca de reprodução de áudio. Dessa forma, ainda que realizar o acionamento simultâneo dos pinos seja uma possível melhoria no projeto, mais efetivo seria usar uma biblioteca mais veloz.

AGRADECIMENTOS

O grupo gostaria de agradecer aos alunos da UTFPR Gustavo Cruz, de Engenharia Mecânica e Gabriel Gil, de Engenharia Mecatrônica, pelo auxílio no design dos pinos; a Juliana Vieira e Luiza Ocampos, alunas do Departamento Acadêmico de Eletrônica (DAELN) e participantes da Metodologia de Ensino Inovador da UTFPR (MEI-U), pelo auxílio no uso de programas "slicer" e impressão 3D; a Ana Militão, Luisa de Melo, Sarah Sezerino, Willian Luiz e, na figura deles, a todos os integrantes do projeto de extensão e comunidade maker "O Projeto" da UTFPR, pelo empréstimo de equipamentos de corte, medição e acabamento e pela ajuda com programas de desenho técnico.

Agradecemos também ao professor Josmael Kampa, responsável pelo Laboratório de Protótipos "PROTO", pelo auxílio na impressão dos pinos móveis e no corte das chapas da MDF, além da disponibilização dos recursos para essas operações; aos professores da disciplina, Eduardo Santos, Juliano Vieira e Ronnier Rohrich, pelo acompanhamento e dicas para o projeto ao longo do semestre e, por fim, à própria UTFPR, pela cessão da licença de uso do SolidWorks e das instalações usadas na realização do projeto.

REFERÊNCIAS

- [1] O Sistema braille e seu uso no Brasil <https://www.gov.br/ibc/pt-br/pesquisa-e-tecnologia/materiais-especializados-1/livros-em-braille-1/o-sistema-braille>
- [2] E.F. Alvaristo et al. *Contribuições do método de Paulo Freire à alfabetização de adultos cegos*. Revista da Faculdade de Educação da UFG, 2021.
- [3] R.D. Batista. *A importância do método braille para a educação na ótica de adultos cegos*. 9ª Mostra Acadêmica UNIMEP, 2011.
- [4] Ministério da Educação (MEC). *Normas técnicas para a produção de textos em braille*. Secretaria de Educação Continuada, Alfabetização, Diversidade e Inclusão – SECADI, 2018.
- [5] J. Melare. *Novas tecnologias facilitam a leitura e o letramento de deficientes visuais*. Sociedade Brasileira para o Progresso da Ciência (SBPC), 2013.
- [6] Datasheet da Raspberry Pi4 Modelo B. <https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-datasheet.pdf>.
- [7] Documentação da Raspberry sobre acesso remoto. <https://www.raspberrypi.com/documentation/computers/remote-access.html>
- [8] Datasheet do servo motor SG90. <https://www.friendlywire.com/projects/ne555-servo-safe/SG90-datasheet.pdf>.
- [9] DataSheet da Impressora 3D Ender3 Max Neo [https://hd-systems.com.ua/_Info/3D_Printers/PDF_Manuals/Ender-3%20Max%20Neo%20-%20User%20Manual%20\(HDS\).pdf](https://hd-systems.com.ua/_Info/3D_Printers/PDF_Manuals/Ender-3%20Max%20Neo%20-%20User%20Manual%20(HDS).pdf)
- [10] Site usado para fazer o design inicial do gabinete <https://pt.makercase.com/#/>
- [11] Tutorial de controle de volume da Raspberry com potenciômetro <https://www.instructables.com/DIY-Raspberry-Pi-Volume-Control-Knob-Using-an-ADS1/>
- [12] Site oficial da linguagem Python <https://www.python.org/>
- [13] Guia HTML <https://developer.mozilla.org/en-US/docs/Web/HTML>
- [14] Código completo do projeto https://github.com/Arthur-Heberle/Oficinas_1